

TravelMate智能旅游助手系统设计说明书

1 引言

1.1 编写目的

本《系统设计说明书》用于对 **TravelMate 智能旅游助手系统** 的软件设计进行规范化描述，包括系统的总体结构、模块划分、数据设计、接口设计、性能与安全设计等内容。

本说明书的目的在于：

- 为开发人员提供统一、完整、准确的设计依据；
- 为测试人员提供测试用例编制和测试依据；
- 为运维人员提供系统部署与维护参考；
- 为项目评审和后续迭代提供可追溯的设计文档。

1.2 项目概述

TravelMate（中文名：旅伴）是基于 **对话式人工智能** 的旅行规划助手，主要目标是**为 18-25 岁学生群体** 及部分年轻上班族提供：

- 一句话自然语言输入需求；
- 在 3 步操作内自动生成完整行程方案；
- 提供 **预算估算、行程管理、地图可视化** 等能力；
- 保持 **零广告、低成本、高个性化** 的体验。

系统采用 **微信小程序 + Python/FastAPI 后端 + 大语言模型（LLM）+ 本地 POI 数据库 + 第三方地图服务** 的技术路线。

1.3 文档读者

本说明书适用于以下角色：

- 系统架构师、后端/前端开发工程师；

- 测试工程师；
- 运维工程师；
- 项目经理及教学指导老师；
- 后续维护与二次开发人员。

1.4 术语与缩写

本说明书中使用的术语与缩写如下表所示：

缩写/术语	英文全称/中文含义	
TM	TravelMate 系统	
LLM	Large Language Model，大语言模型	
API	Application Programming Interface，应用程序编程接口	
REST	Representational State Transfer，一种 Web API 风格	
JSON	JavaScript Object Notation，数据交换格式	
POI	Point of Interest，兴趣点（景点/餐厅/酒店等）	
UI	User Interface，用户界面	
DB	Database，数据库	
MVP	Minimum Viable Product，最小可行产品	
HTTP	HyperText Transfer Protocol，超文本传输协议	
JWT	JSON Web Token，一种身份令牌格式	
TPS	Transactions Per Second，每秒事务数	
QPS	Queries Per Second，每秒请求数	

1.5 参考资料

- 《计算机软件文档编制规范（GB/T 8567）》
- 项目《TravelMate 选题报告》及需求分析 PPT
- 微信小程序官方开发文档
- FastAPI 官方文档
- DeepSeek / GPT 系列模型 API 文档
- 腾讯位置服务、百度地图 API 文档

2 任务概述

2.1 开发目标

通过本项目开发一个 **面向学生群体的智能旅行规划系统**，实现以下目标：

1. 用户可通过微信小程序，以自然语言提交出行需求；
2. 系统基于用户偏好与预算约束，自动生成结构化行程（天/时段/POI）；
3. 提供行程板视图、地图视图、预算视图等多种形式展示；
4. 支持用户对行程进行 **拖拽调整、替换景点、预算实时更新** 等操作；
5. 提供稳定可扩展的后端服务，为后续引入更多“杀手级功能”（如预算锁定行程、实时纠错行程等）预留接口。

2.2 系统主要功能

- **智能对话交互**：自然语言对话、上下文理解、用户偏好记忆；
- **行程一键生成**：基于规则引擎 + LLM 的行程草案自动生成；
- **行程管理与编辑**：行程卡片拖拽排序、替换景点、撤销/重做；
- **预算估算与分解**：总预算估算，按交通/住宿/餐饮/门票分项展示；
- **地图与 POI 展示**：行程地图可视化、POI 标注、路径粗略规划；
- **用户管理与偏好学习**：微信登录、偏好存储、个性化推荐；
- **系统运维与监控**：日志记录、错误告警、性能监控（后续可增强）。

2.3 运行环境

- **客户端环境**
 - 微信小程序运行环境（iOS / Android 微信客户端）
- **服务器环境**
 - 操作系统：Linux（如 Ubuntu 20.04）
 - 运行环境：Python 3.10+，FastAPI
 - Web 服务器：Uvicorn / Gunicorn + Nginx（可选）
 - 数据库：MySQL / PostgreSQL（任选其一）
- **第三方服务**

- LLM: DeepSeek API (主), GPT-3.5-Turbo (备)
- 地图: 腾讯位置服务 (主), 百度地图 API (备)

2.4 设计约束

- 必须符合微信小程序平台规范;
- LLM 调用存在 QPS 和费用限制, 需要设计缓存与降级策略;
- 地图及第三方数据接口受配额和网络延迟影响;
- 数据存储需遵守隐私保护要求 (用户行程和偏好需加密存储);
- 在学校/学生服务器资源有限条件下, 需兼顾性能与成本。

3 总体设计

3.1 设计思想

系统采用 **分层架构 + 微服务化划分思想**, 通过以下方式降低耦合度并提升可扩展性:

- 前后端分离: 微信小程序作为纯前端, 所有业务逻辑由后端 REST API 提供;
- 业务分层: 拆分为用户管理、对话路由、智能体核心、行程引擎、数据服务、地图服务等模块;
- 面向接口设计: 模块之间通过确定的接口协议交互 (HTTP/JSON);
- 可插拔 LLM: 通过统一的 LLM 适配层, 使不同模型 API 可以互换;
- 支持“杀手级功能”扩展: 如预算锁定行程、实时行程纠错等作为独立子模块, 挂载在行程规划引擎之上。

3.2 系统总体结构

3.2.1 分层结构

系统逻辑分层结构如下:

表示层 (Presentation Layer)

- └─ 微信小程序 UI 层
 - └─ 智能对话界面
 - └─ 行程看板界面
 - └─ 地图展示界面

└─ 登录与个人中心界面

业务层 (Service Layer)

- └─ 用户管理服务 (User Service)
- └─ 对话路由服务 (Conversation Router Service)
- └─ 智能体核心服务 (AI Agent Core Service)
- └─ 行程规划引擎 (Itinerary Planning Engine)
- └─ 预算估算服务 (Budget Service)
- └─ 地图集成服务 (Map Integration Service)

数据层 (Data Layer)

- └─ 数据服务层 (Data Service)
 - └─ 本地 POI 数据库访问子模块
 - └─ 用户与行程数据访问子模块
 - └─ 外部 API 代理子模块
- └─ 持久化存储 (DB: MySQL / PostgreSQL)

3.2.2 部署结构

简化部署结构如下：

[用户微信客户端]

|

微信小程序

|

[HTTPS/REST]

|

[应用服务器 (FastAPI)]

└─ 业务服务模块

└─ LLM 适配模块 → [DeepSeek API / GPT-3.5 API]

└─ 地图适配模块 → [腾讯位置服务 / 百度地图]

[数据库服务器]

└─ 用户与行程数据表

└─ POI 数据表

3.3 子系统划分

根据功能与职责，将 TM 系统划分为如下子系统：

1. 用户管理子系统
2. 智能对话与会话管理子系统
3. 行程规划与预算子系统
4. 数据与 POI 管理子系统
5. 地图与位置服务子系统
6. 系统支撑与监控子系统（后续可扩展）

后续章节将逐层展开各子系统设计。

3.4 系统用例概述

核心用例如下：

- UC-01：用户通过微信登录系统；
- UC-02：用户输入自然语言旅行需求；
- UC-03：系统生成行程草案并展示；
- UC-04：用户拖拽调整行程、替换景点；
- UC-05：系统重新估算预算并展示分项；
- UC-06：用户在地图中查看行程路线与 POI；
- UC-07：用户保存行程并可再次加载编辑。

4 功能设计（分层描述）

4.1 用户管理子系统

4.1.1 功能描述

- 完成微信小程序登录态接入；
- 建立用户基础信息（openId、昵称等）；
- 管理用户旅行偏好（预算范围、出行方式偏好、喜好景点类型等）；
- 提供用户信息查询与更新接口。

4.1.2 主要功能模块

- UM-01: 微信登录与认证模块
- UM-02: 用户资料管理模块
- UM-03: 用户偏好管理模块

4.1.3 输入输出

- 输入: 微信 loginCode、用户基本信息、偏好更新请求;
- 输出: 用户标识 (userId)、用户信息、偏好 JSON 数据。

4.2 智能对话与会话管理子系统

4.2.1 功能描述

- 接收用户自然语言输入;
- 维护会话上下文 (历史消息、用户偏好);
- 将请求路由到智能体核心服务与行程引擎;
- 存储对话记录, 用于后续分析和个性化优化。

4.2.2 主要功能模块

- CS-01: 会话管理模块 (Conversation Manager)
- CS-02: 对话路由模块 (Routing)
- CS-03: 消息持久化模块 (Conversation Log)

4.2.3 分层说明

1. **接口层**: 暴露 REST 接口 `/api/conversation/send`, 接收用户消息;
2. **业务层**: 解析消息, 判断意图类型 (规划新行程 / 编辑行程 / 查询行程);
3. **数据层**: 读写 Conversation 表, 持久化消息。

4.3 行程规划与预算子系统

4.3.1 功能描述

- 根据用户自然语言输入 + 用户偏好 + POI 数据, 生成可执行行程草案;
- 提供卡片化的行程数据结构 (按天/时间段/POI);
- 根据行程自动计算预算, 并按类别细分;

- 提供手动调整行程时的预算同步更新能力。

4.3.2 功能模块

- IP-01: 意图分析与需求抽取模块
- IP-02: 行程生成模块（规则引擎 + LLM）
- IP-03: 预算估算模块
- IP-04: 行程调整与同步模块（拖拽、替换、撤销/重做）

4.3.3 典型流程（生成行程草案）

1. 用户输入：“北京三日游，预算 1000 块，想多看看人文景点”；
2. 对话路由服务将消息连同用户偏好传给智能体核心；
3. 智能体核心调用 LLM，生成初步结构化行程；
4. 行程引擎根据本地 POI 数据和规则进行修正（时间合理性、地理相近性等）；
5. 预算模块遍历行程中的各 POI，汇总费用，生成总预算及分项预算；
6. 返回结构化 JSON 给前端。

4.4 数据与 POI 管理子系统

4.4.1 功能描述

- 提供统一的数据访问接口（用户、行程、POI）；
- 封装底层数据库访问逻辑；
- 管理 POI 数据的导入、更新和质量校验；
- 为行程规划与预算提供数据支持。

4.4.2 功能模块

- DS-01: 用户与行程数据访问模块
- DS-02: POI 数据访问模块
- DS-03: 外部数据同步模块（爬虫/定时任务）
- DS-04: 数据质量监控模块（后续增强）

4.5 地图与位置服务子系统

4.5.1 功能描述

- 集成微信小程序地图组件；
- 提供 POI 坐标查询、路线规划（调用腾讯/百度地图 API）；
- 支持在地图上展示行程路径与景点标记；
- 提供基础路径规划优化能力。

4.5.2 功能模块

- MAP-01：地图组件适配模块（前端）
 - MAP-02：地图服务接入模块（后端）
 - MAP-03：位置与路径计算模块
-

5 数据设计

5.1 概念模型

主要实体包括：

- User（用户）
- Conversation（会话）
- Itinerary（行程）
- LocalPoi（本地 POI）

实体关系简述：

- 一个 User 可有多条 Conversation；
- 一个 User 可有多份 Itinerary；
- 一个 Itinerary 由多条行程日（days）组成，其中包含多个 LocalPoi 引用；
- LocalPoi 可被多个 Itinerary 引用。

5.2 主要数据表设计

5.2.1 User 表

User

```
-----
user_id      PK, String
open_id      Unique, String  # 微信 openId
nickname     String
avatar_url   String
travel_preferences JSON      # 偏好配置
created_at   DateTime
updated_at   DateTime
```

5.2.2 Conversation 表

Conversation

```
-----
conversation_id PK, String
user_id        FK → User.user_id
user_message   Text
agent_response  Text
context_snapshot JSON      # 可选：存储上下文摘要
timestamp      DateTime
```

5.2.3 Itinerary 表

Itinerary

```
-----
itinerary_id  PK, String
user_id       FK → User.user_id
title         String
days         JSON      # 结构化行程数据
total_budget  Double
currency      String    # 默认 CNY
created_at    DateTime
updated_at    DateTime
```

行程 JSON 示例：

```
{
  "days": [
    {
      "date": "2025-11-01",
      "activities": [
        {"time": "09:00", "poild": "POI_001", "type": "景点"},
        {"time": "12:00", "poild": "POI_010", "type": "餐饮"}
      ]
    }
  ],
  "totalBudget": 950
}
```

5.2.4 LocalPoi 表

LocalPoi

poi_id	PK, String	
name	String	
type	String	# 景点/餐厅/酒店等
city	String	
lat	Double	# 纬度
lng	Double	# 经度
estimated_cost	Double	
tags	JSON	# 如 ["人文", "博物馆", "适合学生"]
source	String	# 数据来源
updated_at	DateTime	

5.3 数据存储与访问策略

- 所有关系型数据使用 MySQL/PostgreSQL 存储；
- 高频查询（热门城市、热门 POI）可考虑后续引入缓存（如 Redis）；
- 行程 days 字段采用 JSON 存储，方便与前端直接交互；
- 对敏感数据（如用户偏好）在 DB 层进行加密存储。

6 接口设计（概要）

6.1 前后端接口规范

- 协议：HTTPS + REST
- 数据格式：JSON
- 认证方式：微信登录 + 后端会话管理（可扩展为 JWT）

6.1.1 示例接口：发送对话消息

- URL： `POST /api/conversation/send`
- 请求体：

```
{
  "userId": "U123",
  "conversationId": "C456",
  "message": "北京三日游，预算1000块"
}
```

- 响应体（简化示例）：

```
{
  "conversationId": "C456",
  "reply": "好的，为你规划北京三日游，总预算控制在1000元左右.....",
  "itineraryPreview": { ... },
  "status": "ok"
}
```

6.1.2 示例接口：获取行程详情

- URL： `GET /api/itinerary/{itineraryId}`
- 响应体：

```
{
  "itineraryId": "I001",
  "title": "北京三日游-学生预算版",
  "days": [ ... ],
}
```

```
"totalBudget": 960.0
}
```

6.2 LLM 接口适配设计

引入统一的 LLM 适配层：

- 对外暴露：`LLMService.generateItinerary(prompt, context)`
- 内部可根据配置选择：
 - DeepSeek 接口；
 - GPT-3.5-Turbo 接口；
 - 模板化降级方案。

6.3 地图服务接口设计

- 封装 `MapService`，对上层提供：
 - `getPoiCoordinate(poiName, city)`
 - `getRoute(startCoord, endCoord)`
- 内部根据当前可用性选择腾讯或百度地图。

7 关键模块详细设计（示例层次）

7.1 对话路由服务（Conversation Router Service）

7.1.1 职责

- 接收来自前端的对话请求；
- 解析请求并识别意图（规划新行程 / 编辑行程 / 查询行程）；
- 将请求分发到相应业务模块；
- 统一封装响应。

7.1.2 处理流程

1. 接收 HTTP 请求
2. 校验 `userId` / `conversationId`

3. 查询用户偏好 (User.travel_preferences)
4. 调用意图识别 (可由 LLM 或简单规则)
5. 根据意图:
 - NEW_TRIP → 调用行程规划引擎
 - EDIT_TRIP → 调用行程调整模块
 - QUERY_TRIP → 读取 Itinerary 数据
6. 记录 Conversation
7. 返回统一响应结构

7.2 行程规划引擎 (Itinerary Planning Engine)

7.2.1 职责

- 将自然语言需求转化为结构化行程；
- 根据 POI 数据与规则校正模型输出；
- 与预算模块和地图模块协同工作。

7.2.2 设计分层

- **需求解析层**：抽取出发地、目的地、出行天数、预算上限、偏好标签等；
- **候选 POI 选取层**：根据城市+偏好，从 LocalPoi 中选取候选集；
- **行程构造层**：使用规则（时间/距离约束）+ LLM 调整生成合理日程；
- **预算计算层**：调用预算服务汇总费用。

7.3 预算估算模块 (Budget Service)

7.3.1 职责

- 根据行程中选定的 POI，计算总预算；
- 将预算拆分为交通/住宿/餐饮/门票等类别；
- 支持在行程修改后实时重新计算。

7.3.2 计算逻辑 (简要)

总预算 = Σ (景点门票费用) + Σ (餐饮估计费用) + Σ (住宿估计费用) + 简化交通费用

- 费用源自 LocalPoi.estimated_cost;
 - 对于住宿，可按“每晚 × 天数”估算；
 - 交通费用可按“城市内平均值 × 出行天数”简化处理（后续可细化）。
-

8 性能与可靠性设计

8.1 性能指标

- 对话响应时间：< 5 秒（依赖 LLM 调用）；
- 行程生成时间：< 10 秒；
- 界面操作响应：< 1 秒（以小程序前端为主）；
- 地图加载时间：< 3 秒（在网络良好条件下）。

8.2 性能优化思路

- 使用异步编程（FastAPI + async/await）提高并发；
- 对常用 POI 和热门城市做缓存；
- 对 LLM 结果进行适度缓存（如相似需求模板）；
- 部署时通过 Nginx 反向代理 + 多 worker 模式提升吞吐。

8.3 可靠性与降级策略

- 当 LLM 接口不可用时：
 - 启用模板化行程生成（简单规则）；
 - 给出友好提醒：“当前智能规划服务受限，提供基础行程建议。”
 - 当地图服务不可用时：
 - 前端保留行程列表视图；
 - 提示用户稍后查看地图；
 - 对关键操作（保存行程）进行失败重试和事务保护。
-

9 安全性设计

9.1 身份验证

- 使用微信登录态作为基础认证机制；
- 后端记录 userId 与 openId 的绑定关系；
- 所有操作需在登录态下进行。

9.2 权限控制

- 普通用户仅能访问自身行程和对话数据；
- 管理后台（如有）需单独的管理员认证方式；
- 后端接口对 userId 与资源绑定进行校验。

9.3 数据安全与隐私保护

- 对用户行程和偏好数据存储加密（如字段级加密）；
 - 传输层使用 HTTPS 保证数据在传输过程中的安全；
 - 对日志进行脱敏处理，不记录敏感信息的明文。
-

10 出错处理与日志

10.1 出错处理原则

- 所有接口返回统一错误结构（包括错误码和信息）；
- 前端展示友好提示信息，不暴露系统内部细节；
- 对于可重试错误（网络波动、第三方接口超时）进行重试。

10.2 日志记录

- 记录重要业务操作日志（登录、生成行程、保存行程等）；
 - 记录系统运行日志（错误、警告、第三方调用异常）；
 - 日志分级（INFO/WARN/ERROR），便于定位问题。
-

11 配置与可扩展性

- 使用配置文件/环境变量管理：

- LLM API Key、地图 API Key；
 - 不同环境（开发/测试/生产）的 DB 配置；
 - 可插拔扩展：
 - LLM 供应商切换；
 - 新增“杀手级功能”模块（如预算锁定行程、实时天气驱动行程调整），通过在行程引擎中增加策略层实现。
-